

University of Haifa Pre-Slurm Tutorial

Introduction:

The best practice of running a job in Slurm environment is to use [NVIDIA Enroot](#), pulling an optimized container from [NVIDIA NGC](#), modifying it according to the user environment and code, converting the container to a “squash” file and finally submitting a Slurm job based on the modified container and the mounted code.

This documentation includes a basic tutorial for achieving this procedure, with an example of running a TensorFlow / PyTorch code using [Horovod](#).

Note: this tutorial assumes your environment includes packages not installed or with different versions from the ones arriving with the NGC container. If this is not relevant you can skip modifying the container before running it and just use the container as it is.

Procedure:

1. **NVIDIA Enroot** – install [NVIDIA Enroot](#) if not yet installed.
2. **Code** – use your own code, or clone [Horovod](#) GitHub repository for the example (i.e., by running `git clone https://github.com/horovod/horovod`).
3. **Environment packages** – prepare a requirements file with your current environment packages to later update the container:
 - a. Run `pip freeze > requirements.txt` to export your current environment to a file.
 - b. Run `grep '^<torch|tensorflow>==' requirements.txt` to print your current PyTorch / TensorFlow. You will use this information in step 4.
 - c. Run `grep -v '^<torch|tensorflow>==' requirements.txt > temp && mv temp requirements.txt` to delete the line which includes the PyTorch / TensorFlow version, as we don't want to later override the container's version.
 - d. Optional: run `echo '<package>==<version>' >> requirements.txt` to add any other relevant packages. E.g., for the Horovod's PyTorch example run `echo 'horovod==0.23.0' >> requirements.txt`.
4. **NGC container** – pull the relevant PyTorch / TensorFlow container from NVIDIA NGC:
 - a. Visit NVIDIA documentation for all [NGC PyTorch containers release notes](#) or [NGC TensorFlow containers release notes](#).
 - b. Click on the PyTorch / TensorFlow release notes of the latest version.
 - c. Scroll down to the “NVIDIA PyTorch / TensorFlow Container Versions” section and search for the container version which fits the PyTorch / TensorFlow version you printed in 3b.
 - d. Run `enroot import 'docker://nvcr.io#nvidia/pytorch:<version>-py3'` for PyTorch or `enroot import 'docker://nvcr.io#nvidia/tensorflow:<version>-tf1-py3'` for

TensorFlow with the relevant version from 4c, this will pull the PyTorch / TensorFlow container from NGC and create a squash file out of it in your local directory. For the Horovod's example use PyTorch version 21.08 or TensorFlow version 22.01.

5. **Modify the container** – skip this step if running Horovod's TensorFlow example, as it already contains Horovod installed. Temporarily start the PyTorch / TensorFlow container to modify its packages according to yours (note how this step includes creating, starting, and exporting the container):
 - a. Run `enroot create --name nvidia_<pt|tf> <squash file>` to unpack the content of the squash file to Enroot's data path. Run `enroot list` to see all unpacked containers.
 - b. Run `enroot start --root --rw --mount ./mount nvidia_<pt|tf>` to start the container from Enroot's data path with read and write permissions and your current environment mounted, so you will have access to the "requirements.txt" file.
 - c. Run `pip install -r /mount/requirements.txt` to install and update the container environment based on your environment.
 - d. Optional: run your code while inside the container. E.g., to run Horovod's PyTorch synthetic data example run `python /mount/horovod/examples/pytorch/pytorch_synthetic_benchmark.py`.
 - e. Run `exit` to exit the container.
 - f. Run `enroot export --output nvidia_<pt|tf>.sqsh nvidia_<pt|tf>` to export the modified unpacked container with your changes back to a local squash file.
 - g. Optional: remove the local container from Enroot's data path by running `enroot remove nvidia_<pt|tf>` and delete the squash file of the original container, as we don't need them anymore.
6. **Accessing the container and your code** – in case there's no access from the cluster to your home directory, to the squash file or to your code, move the relevant files to a directory available to the cluster.
7. **Submit a Slurm job** – run a Slurm job based on the modified container while mounting your code.
 - a. For running Horovod's example on two nodes with 8 GPUs each create a new script file by running `vim horovod.sub`, paste in the following code and save the file:

```
#!/bin/bash
#SBATCH --job-name horovod_<pt|tf>
#SBATCH --nodes 2
#SBATCH --gres gpu:8
#SBATCH --ntasks-per-node 8
#SBATCH --cpus-per-task 32

srun --container-image $1 \
    --container-mounts $2:/code \
    --no-container-entrypoint \
    /bin/bash -c \
```

```
"python  
/code/horovod/examples/<pytorch|tensorflow>/<pytorch|tensorflow>_  
synthetic_benchmark.py \  
--batch-size 256"
```

- b. Submit a new Slurm job by running `sbatch horovod.sub <path to the modified container squash file> <path to the directory with Horovod's code>`.
 - i. Note: add `-w <node name #1>,<node name #2>` after `sbatch` to choose specific nodes.
- c. Optional: attach to the submitted job by running `sattach <job_id>.0`. Additionally, a file with the job output should be created in the directory where you submitted the job. To detach the submitted job press `Ctrl+Z`.